

Sheet 3

نور إيهاب سمير محمد 20241063

global.h

```
#ifndef GLOBAL_H
#define GLOBAL_H
#define MAX_SIZE 100
typedef struct customer_data {
    int id;
    char name[50];
} Customer;
typedef Customer Type;
#endif
```

stack.h

```
#ifndef STACK_H
#define STACK_H
#include "global.h"
typedef struct stack {
    int top;
    Type item[MAX_SIZE];
} Stack;
void createStack(Stack *s);
```

```
int isEmpty(Stack s);  
int isFull(Stack s);  
void push(Type item, Stack *s);  
Type pop(Stack *s);  
#endif
```

stack.c

```
#include <stdio.h>  
#include "stack.h"  
void createStack(Stack *s) {  
    s->top = -1;  
}  
int isEmpty(Stack s) {  
    return s.top == -1;  
}  
int isFull(Stack s) {  
    return s.top == MAX_SIZE - 1;  
}  
void push(Type item, Stack *s) {  
    if (!isFull(*s)) s->item[++s->top] = item;  
}  
Type pop(Stack *s) {  
    Type item = {0, ""};  
    if (!isEmpty(*s)) item = s->item[s->top--];  
    return item;  
}
```

queue.h

```
#ifndef QUEUE_H
#define QUEUE_H
#include "global.h"

typedef struct queue {
    int front;
    int rear;
    Type item[MAX_SIZE];
} Queue;

void createQueue(Queue *q);
int isQueueempty(Queue q);
int isQueuefull(Queue q);
void enqueue(Type item, Queue *q);
Type dequeue(Queue *q);
Type firstElement(Queue q);
Type lastElement(Queue q);
void destroyQueue(Queue *q);
void copyQueue(Queue *src, Queue *dest);
int queueSize(Queue q);

#endif
```

queue.c

```
#include <stdio.h>
#include "queue.h"

void createQueue(Queue *q) {
    q->front = 0;
```

```

    q->rear = 0;
}

int isEmpty(Queue q) {
    return q.front == q.rear;
}

int isQueueFull(Queue q) {
    return (q.rear + 1) % MAX_SIZE == q.front;
}

void enqueue(Type item, Queue *q) {
    if (!isQueueFull(*q)) {
        q->rear = (q->rear + 1) % MAX_SIZE;
        q->item[q->rear] = item;
    } else {
        printf("Queue Overflow\n");
    }
}

Type dequeue(Queue *q) {
    Type item = {0, ""};
    if (!isEmpty(*q)) {
        q->front = (q->front + 1) % MAX_SIZE;
        item = q->item[q->front];
    }
    return item;
}

Type firstElement(Queue q) {
    if (!isEmpty(q)) return q.item[(q.front + 1) % MAX_SIZE];
}

```

```

    Type empty = {0, ""};
    return empty;
}

Type lastElement(Queue q) {
    if (!isEmpty(q)) return q.item[q.rear];
    Type empty = {0, ""};
    return empty;
}

void destroyQueue(Queue *q) {
    q->front = q->rear = 0;
}

int queueSize(Queue q) {
    return (q.rear - q.front + MAX_SIZE) % MAX_SIZE;
}

void copyQueue(Queue *src, Queue *dest) {
    createQueue(dest);
    int i = (src->front + 1) % MAX_SIZE;
    while (i != (src->rear + 1) % MAX_SIZE) {
        enqueue(src->item[i], dest);
        i = (i + 1) % MAX_SIZE;
    }
}

```

main.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```

#include <string.h>

#include "stack.h"

#include "queue.h"

#include "global.h"

Customer getCustomerData() {
    Customer c;

    printf("Enter Customer ID: ");

    scanf("%d", &c.id);

    while(getchar() != '\n');

    printf("Enter Customer Name: ");

    fgets(c.name, 50, stdin);

    c.name[strcspn(c.name, "\n")] = 0;

    return c;
}

void printCustomer(Customer c) {
    printf("[ID: %d | Name: %s]\n", c.id, c.name);
}

void displayQueueCustomers(Queue q) {
    if (isEmpty(q)) {
        printf("Queue is empty.\n");

        return;
    }

    int i = (q.front + 1) % MAX_SIZE;

    int count = 1;

    while (i != (q.rear + 1) % MAX_SIZE) {
        printf("%d. ", count++);
    }
}

```

```

        printCustomer(q.item[i]);
        i = (i + 1) % MAX_SIZE;
    }
}

void displayMostRecentOrder(Queue q) {
    if (isEmpty(q)) {
        printf("Queue is empty.\n");
        return;
    }
    Stack s;
    createStack(&s);
    int i = (q.front + 1) % MAX_SIZE;
    while (i != (q.rear + 1) % MAX_SIZE) {
        push(q.item[i], &s);
        i = (i + 1) % MAX_SIZE;
    }
    int count = 1;
    while (!isEmpty(s)) {
        printf("%d. ", count++);
        printCustomer(pop(&s));
    }
}

int main() {
    Queue customerQueue;
    createQueue(&customerQueue);
    int choice;

```

```

do {
    printf("\n===== Car Workshop Customer Management =====\n");
    printf("1. Add a New Customer\n");
    printf("2. Serve a customer\n");
    printf("3. Display Customers Information\n");
    printf("4. Display Customers in Most-Recent Order\n");
    printf("5. Exit\n");
    printf("Enter choice: ");
    if (scanf("%d", &choice) != 1) {
        while(getchar() != '\n');
        continue;
    }
    switch(choice) {
        case 1: {
            Customer newCustomer = getCustomerData();
            enqueue(newCustomer, &customerQueue);
            printf("Customer added successfully.\n");
            break;
        }
        case 2: {
            if (!isEmpty(customerQueue)) {
                printf("Serving customer: ");
                printCustomer(dequeue(&customerQueue));
            } else printf("Queue is empty.\n");
            break;
        }
    }
}

```

```
        case 3: displayQueueCustomers(customerQueue); break;
        case 4: displayMostRecentOrder(customerQueue); break;
        case 5: printf("Exiting program.\n"); break;
        default: printf("Invalid choice.\n"); break;
    }
} while(choice != 5);
return 0;
}
```